LuciOS — Project Proposal

Version 1.1 · Revised October 22, 2026
Prepared for: Luci Crow
Type: Interactive professional technology portfolio

> **Design update:** The first release uses a hanging overhead app conveyor and one mechanical holographic display, and showcases **real screenshots and code**, rather than a radial launcher or fake project applications. Mystery/puzzle concepts remain **long-term ideas only**.

1. Executive concept

LuciOS transforms a technology portfolio into a browser-based fictional operating system. Its purpose is to show what Luci can build while presenting credible, easy-to-find evidence of her programming, networking, troubleshooting, and web development experience. The experience pairs **futuristic cyberpunk holograms** with **steampunk machinery**: metal, gears, pipes, valves, gently moving mechanical objects and a black/pastel-pink/grey/white palette. The atmosphere can be eerie, but the content stays clear and professional.

The main interaction is not a desktop menu. Application tags hang from a **rotating overhead conveyor**, reminiscent of a dry-cleaner's garment rail. The visitor selects one; a mechanical hand can retrieve it, a blank screen lowers over the desktop, bronze piping frames it, and holographic application content activates inside. This becomes the project's distinctive navigation metaphor.

2. Audience and objectives

Audience	Desired result
Employers / internship supervisors	Quickly access skills, resume, contact and verified project evidence
Instructor	Confirm every course portfolio component is present and functional
Technical reviewers	Inspect code choices, contributions, repositories and lessons learned
Curious visitors	Explore an inventive interface without hiding essential content
Portfolio owner	Update screenshots, projects and resume without rewriting the UI

Outcome: A publicly hosted website with at least three supported technical projects, real documentation, a resume, safe contact information and practical accessibility. LuciOS itself should be a fourth demonstration of UI architecture and creative problem solving.

3. First release — LuciOS core

Entrance and desktop

The site begins with a brief LuciOS progress bar followed by a large, raised **red Enter button on a black background**. It is a dramatic start screen, not authentication. After Enter, visitors see a quiet, somewhat spooky animated wallpaper, pixel application tags on an overhead rail and unobstructed links to the core portfolio. Users can hover/focus the rail ends to scroll tags, then click or press Enter to select an app.

Application presentation

The app tag swings lightly; a mechanical hand can collect it as a progressive enhancement. A large **black app display screen** descends over the desktop, framed by bronze pipes. At the top-right, one pipe continues upward offscreen. Gears briefly turn, a controlled steam effect may vent during loading, and the selected tag is placed into the machine. A pink-edged holographic panel reveals the content inside the frame. **One active app frame** is the v1.1 default, avoiding a complex draggable multi-window

system. A clear Close/Back action returns to the rail. Content must be accessible even if motion is reduced or JavaScript animations do not run.

Core portfolio applications

- **About:** 100–250 word professional biography, academic focus, career goals and background.
- **Projects:** Battleship, CorvusDesk and Bad Apple Repair; three authentic app/site screenshots **per project**, several selected examples of code Luci worked on with explanation, and a verified **direct GitHub repository link below each project**. Include purpose, contributions, stack, decisions and lessons.
- **Skills:** Accurately represented programming languages, cloud, networking, repair, databases and tools.
- **Resume:** Current downloadable PDF; accessible opening path.
- **Contact:** Safe professional links/contact method, optional LinkedIn.
- **Terminal:** A restricted local command interpreter. Valid commands travel through a pipe while the valve briefly releases steam **only during processing**; the result exits the pipe. Incorrect commands shake and show a readable error, without steam.
- **Portfolio View:** Plain, conventional access to all required content without mechanical interactions.

Content safeguards

Only use real, approved screenshots/code excerpts and confirmed public repository links. The site's project cards are **not recreated dummy apps**. Do not expose customer data, unreleased employer work, credentials, or private repositories. Avoid invented claims of project completion, results, proficiency or published links.

4. Technology approach

Layer	Proposed approach
Web client	React + TypeScript + Vite
Style / effects	Modular CSS, CSS variables and SVG; animation only where it improves interaction
UI state	App selection, conveyor position, loader phases, current display and terminal phases
Content	Structured local TypeScript/JSON records for bio, projects, screenshots, code highlights, links
Deployment	HTTPS static hosting, likely GitHub Pages or another compatible host
Backend	**Not required for v1.1**; future APIs only for justified new services
Accessibility	Semantic headings, keyboard operation, reduced-motion, readable contrast, alt text, Portfolio View

The app rail and loader are client-side simulations; they do not connect to a real OS. The Terminal is also a browser-local simulation with a strict command allowlist. Optional local storage may retain harmless preferences; no passwords or sensitive personal data should be stored.

5. Delivery strategy for a short deadline

Build the **complete and accurate portfolio content first**, then the original mechanical presentation, then optional animation polish. An accessible no-animation path is part of the initial build, not a late fix.

Phase	Deliverable	Gate
A · Scaffold	React/TypeScript/Vite, tokens, typed content, safe build	Build runs; no fabricated content
B · Portfolio	About, Skills, three authentic project pages, resume, contact and Portfolio View	Instructor-required content present

Phase	Deliverable	Gate
C · Mechanical UI	Boot, red Enter, overhead hanging rail, single display screen and direct shortcuts	Selection opens correct app without animation
D · Theatrical polish	Swing, mechanical hand, pipes, gears, hologram, terminal valve	Effects accessible, interruptible and stable
E · Release	QA, live HTTPS URL, repository README, reflection	Links, permissions, keyboard and download checks pass

The most elaborate arm/grip animation can be simplified if time becomes constrained. An immersive transition is not more important than working professional content.

6. Final core deliverables

- Public portfolio URL, responsive enough for readable access on narrow screens, with desktop-first art direction.
- Public-safe About, Skills, Resume and Contact content plus 100–250 word assignment reflection.
- Three real project case studies, each with three screenshots, favorite code highlights, a **specific** working GitHub repo link and an explanation of actual work.
- Functional overhead app rail and one mechanical/holographic display frame, with a conventional navigation alternative.
- Safe terminal simulation and nonessential processing animations.
- Source repository, documented build steps, UI/UX documentation, SRS, test checks and clean deployment.

7. Longer-term creative vision — excluded from v1.1

The initial concept also imagined hidden system clues, a small interactive garden, a folded upper-right page corner that reveals **evilLuciOS**, an alternate eerie OS with its own puzzles, and an intentionally anticlimactic finish: **“Don't know why but THE END.”** These ideas remain in the project vision but are **not requirements, code tasks, or dependencies for the upcoming portfolio release**. A later release can design them separately after the course deadline.

8. Success criteria and open decisions

A first-time visitor can find Resume, Contact and a complete project case study without completing an animation; screenshots and code are authorized and real; three GitHub links work; essential keyboard and reduced-motion paths function; the deployed site loads reliably and fulfills the instructor's rubric. Still pending: actual project assets, verified URLs, current resume, contact route, hosting choice, and final motion polish.

Project principle: The machine attracts attention; the evidence earns trust.